

DRAFT Scheme of Valuation/Answer Key			
(Scheme of evaluation (marks in brackets) and answers of problems/key)			
APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY			
SIXTH SEMESTER B.TECH DEGREE (R,S)EXAMINATION, May/July 2023(2019 Scheme)			
Course Code: CST302			
Course Name: COMPILER DESIGN			
Max. Marks: 100			Duration: 3 Hours
PART A			
		Answer all questions, each carries 3 marks.	Marks
1		Input buffering definition – 1.5 marks Relevance in lexical analysis – 1.5 marks Lexical analyzer will detect the tokens from the source language with the help of input buffering. The reason is, the lexical analyzer will scan the input character by character, it will increase the cost file read operation. So buffering is used. The buffer is a pair in which each half is equal to system read command.	(3)
2		Diagram – 3 marks <pre>graph LR 0((0)) -- "Start
LE)" --> 0 0 -- "<" --> 1((1)) 1 -- "=" --> 2((2)) 1 -- ">" --> 3((3)) 1 -- "Other" --> 4((4)) 0 -- "=" --> 5((5)) 0 -- ">" --> 6((6)) 5 -- "return(relop, NE)" --> 5 6 -- "=" --> 7((7)) 6 -- ">" --> 8((8)) 2 -- "return" --> 2</pre>	(3)
3		Definition – 1 mark Left recursion removal steps – 2 marks	(3)
4		Definition – 1.5 marks Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive parsing. The basic idea is that when it is not clear which of two alternative productions to use to expand a nonterminal “A”, we may be able to rewrite the “A” productions to refer the decision until we have seen enough of the input to	(3)

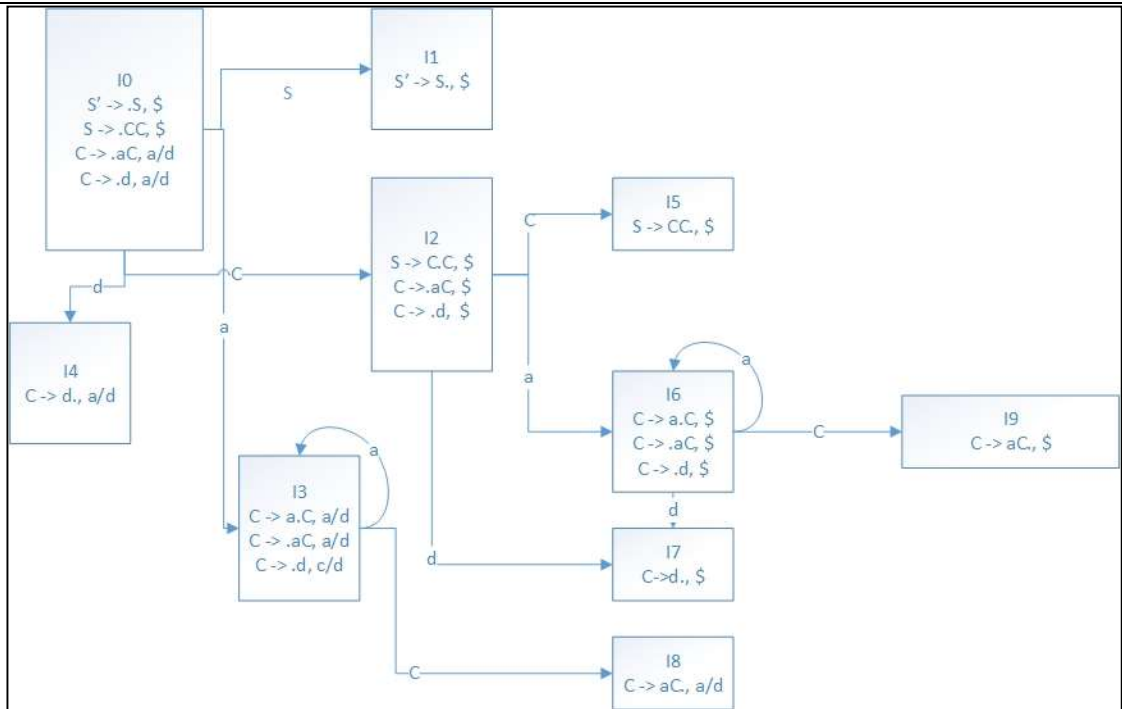
	make the right choice. Left factoring – 1.5 marks $E \rightarrow E + T \mid T$ $T \rightarrow \text{float } T' \mid (E)$ $T' \rightarrow * T \mid \epsilon$																																								
5	Actions – Shift, Reduce, Accept, Error - 1.5 marks Explanation – 1.5 marks	(3)																																							
6	Comparison (any three) - 3 marks <table><tr><td>CLR</td><td>LALR</td></tr><tr><td>It is expensive and difficult to implement.</td><td>It is easy and cheap to implement.</td></tr><tr><td>CLR Parser is the largest in size. As the number of states is very large.</td><td>LALR Parser is the largest in size. Because LALR has less number of states.</td></tr><tr><td>Error detection can be done immediately in CLR Parser.</td><td>Error detection is not immediate in LALR.</td></tr><tr><td>It is very powerful and works on a large class of grammar.</td><td>It is intermediate in power between SLR and CLR</td></tr></table>	CLR	LALR	It is expensive and difficult to implement.	It is easy and cheap to implement.	CLR Parser is the largest in size. As the number of states is very large.	LALR Parser is the largest in size. Because LALR has less number of states.	Error detection can be done immediately in CLR Parser.	Error detection is not immediate in LALR.	It is very powerful and works on a large class of grammar.	It is intermediate in power between SLR and CLR	(3)																													
CLR	LALR																																								
It is expensive and difficult to implement.	It is easy and cheap to implement.																																								
CLR Parser is the largest in size. As the number of states is very large.	LALR Parser is the largest in size. Because LALR has less number of states.																																								
Error detection can be done immediately in CLR Parser.	Error detection is not immediate in LALR.																																								
It is very powerful and works on a large class of grammar.	It is intermediate in power between SLR and CLR																																								
7	<table><tr><td><u>Three Address Code</u></td><td colspan="3"><u>Quadruple</u></td></tr><tr><td></td><td><u>Op</u></td><td><u>S1</u></td><td><u>S2</u></td><td><u>Destination</u></td></tr><tr><td>t1 = uminus c</td><td>-</td><td>c</td><td></td><td>t1</td></tr><tr><td>t2 = b * t1</td><td>*</td><td>b</td><td>t1</td><td>t2</td></tr><tr><td>t3 = uminus c</td><td>-</td><td>c</td><td></td><td>t3</td></tr><tr><td>t4 = b * t3</td><td>*</td><td>b</td><td>t3</td><td>t4</td></tr><tr><td>t5 = t2 + t4</td><td>+</td><td>t2</td><td>t4</td><td>t5</td></tr><tr><td>a = t5</td><td>=</td><td>a</td><td></td><td>t5</td></tr></table> Three Address code conversion steps – 1.5 marks Conversion to quadruple -1.5 marks	<u>Three Address Code</u>	<u>Quadruple</u>				<u>Op</u>	<u>S1</u>	<u>S2</u>	<u>Destination</u>	t1 = uminus c	-	c		t1	t2 = b * t1	*	b	t1	t2	t3 = uminus c	-	c		t3	t4 = b * t3	*	b	t3	t4	t5 = t2 + t4	+	t2	t4	t5	a = t5	=	a		t5	(3)
<u>Three Address Code</u>	<u>Quadruple</u>																																								
	<u>Op</u>	<u>S1</u>	<u>S2</u>	<u>Destination</u>																																					
t1 = uminus c	-	c		t1																																					
t2 = b * t1	*	b	t1	t2																																					
t3 = uminus c	-	c		t3																																					
t4 = b * t3	*	b	t3	t4																																					
t5 = t2 + t4	+	t2	t4	t5																																					
a = t5	=	a		t5																																					
8	Definition – 2 marks Syntax directed definition specifies the values of attributes by associating semantic rules with the grammar productions. It is a context free grammar with attributes and rules together which are associated with grammar symbols and productions respectively.	(3)																																							

		Example - 1 marks $E \rightarrow E1 + T \{ E.val = E1.val + T.val \}$									
9		Explanation with example – 3 marks The expression or sub-expression that has been appeared and computed before and appears again during the computation of the code is the common sub-expression. Elimination of that sub-expression is known as Common sub-expression elimination . 1. Local Common Sub-expression elimination – It is used within a single basic block. Where a basic block is a simple code sequence that has no branches. 2. Global Common Sub-expression elimination – It is used for an entire procedure of common sub-expression elimination. Eg: <table> <tr> <td>Before elimination</td> <td>After elimination</td> </tr> <tr> <td>x= 10;</td> <td>x = 10;</td> </tr> <tr> <td>y = x + 1 * 2;</td> <td>y = x + 1 * 2;</td> </tr> <tr> <td>z = x + 1 * 2;</td> <td>w = y + x;</td> </tr> </table> // 'z' has common expression as 'y' w = z + x; (If example is not given then give 1.5 marks.)	Before elimination	After elimination	x= 10;	x = 10;	y = x + 1 * 2;	y = x + 1 * 2;	z = x + 1 * 2;	w = y + x;	(3)
Before elimination	After elimination										
x= 10;	x = 10;										
y = x + 1 * 2;	y = x + 1 * 2;										
z = x + 1 * 2;	w = y + x;										
10		Peephole optimization definition – 1.5 marks Peephole optimization is a type of code Optimization performed on a small part of the code. It is performed on a very small set of instructions in a segment of code. Techniques – 1.5 marks 1. Redundant load and store elimination 2. Constant folding 3. Strength Reduction 4. Null sequences/ Simplify Algebraic Expressions 5. Combine operations 6. Deadcode Elimination (Any three methods)	(3)								
PART B											
Answer one full question from each module, each carries 14 marks.											
		Module I									
11	a)	Phases – 4 marks	(8)								

		Source Program Target Program																
		Explanation using the given example - 4																
	b)	Definition – 3 marks (1 mark for each) <u>Tokens</u>- Sequence of characters that have a collective meaning. <u>Patterns</u>- There is a set of strings in the input for which the same token is produced as output. This set of strings is described by a rule called a pattern associated with the token . <u>Lexeme</u>- A sequence of characters in the source program that is matched by the pattern for a token. Explanation with example – 3 marks <table><thead><tr><th>Token</th><th>Pattern</th><th>Sample Lexeme</th></tr></thead><tbody><tr><td>while</td><td>while</td><td>while</td></tr><tr><td>relation_op</td><td>= != < ></td><td><</td></tr><tr><td>integer</td><td>(0-9)*</td><td>42</td></tr><tr><td>string</td><td>Characters between " "</td><td>"hello"</td></tr></tbody></table>	Token	Pattern	Sample Lexeme	while	while	while	relation_op	= != < >	<	integer	(0-9)*	42	string	Characters between " "	"hello"	(6)
Token	Pattern	Sample Lexeme																
while	while	while																
relation_op	= != < >	<																
integer	(0-9)*	42																
string	Characters between " "	"hello"																
		OR																
12	a)	1. Parse generator 2. Scanner generators 3. Syntax-directed 4. Translation engines	(6)															

		5. Automatic code generator 6. Data flow engines.	
	b)	Buffer pairs – 4 marks A specialized buffering technique can decrease the amount of overhead, which is needed to process an input character in transferring characters. It includes two buffers, each includes N-character size which is reloaded alternatively.	

		called ambiguous grammar. Removal – 4 marks E-> E+T E-T T T-> T * P T/P P P-> P ^ F F F-> (E) id																									
	b)	Removing left recursion – 1 mark First -1 mark Follow – 1 mark Parsing table – 3 marks <table border="1"> <tr> <td></td> <td>(</td> <td>)</td> <td>a</td> <td>,</td> <td>\$</td> </tr> <tr> <td>S</td> <td>S-> (L)</td> <td></td> <td>S->a</td> <td></td> <td></td> </tr> <tr> <td>L</td> <td>L->SL'</td> <td></td> <td>L->SL'</td> <td></td> <td></td> </tr> <tr> <td>L'</td> <td></td> <td>L' →ε</td> <td></td> <td>L->,SL'</td> <td></td> </tr> </table> Proof – 1 mark Here, we can see that there are no two productions in the same cell. Hence, this grammar is an LL(1) Parser.		(	)	a	,	\$	S	S-> (L)		S->a			L	L->SL'		L->SL'			L'		L' →ε		L->,SL'		(7)
	(	)	a	,	\$																						
S	S-> (L)		S->a																								
L	L->SL'		L->SL'																								
L'		L' →ε		L->,SL'																							
		Module III																									
15	a)	Parser Explanation – 6 marks Definition Data structures used Possible Actions 2 Conflicts – 2 marks (2 marks each) 1. A shift-reduce conflict occurs in a state that requests both a shift action and a reduce action. 2. A reduce-reduce conflict occurs in a state that requests two or more different reduce actions.	(8)																								
	b)	Steps- 4 marks	(6)																								



Correct answer - 2 marks

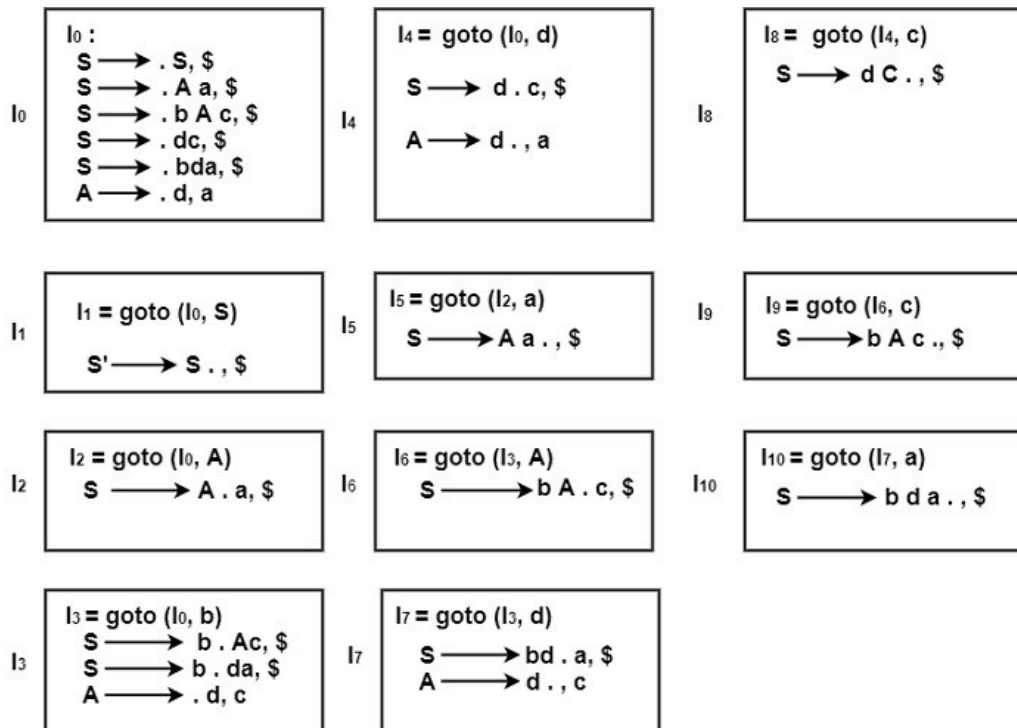
OR

16

Augmented grammar -1 mark

LR(1) items/ Flow diagram – 4 marks

(14)



Parsing table – 4

States	Action							goto
	a	b	c	d	\$	S	B	
0		s3		s4		1	2	
1					accept			
2	s5							
3				s7			6	
4	r5		s8					
5								
6			s9					
7	s10		r5					
8					r3			
9					r2			
10					r4			

Parsing of String "bdc" – 5

Stack	Input String	Action
\$ 0	bdc \$	Shift 3
\$ 0 b 3	dc \$	Shift 7
\$ 0 b 3 d 7	c \$	Reduce by A → d
\$ 0 b 3 A 6	c \$	Shift 9
\$ 0 b 3 A 6 c 9	\$	Reduce by S → b Ac
\$ 0 s 1	\$	accept

Module IV

17 a) 2 marks for each

Quadruple – It is a structure which consists of 4 fields namely op, arg1, arg2 and result. op denotes the operator and arg1 and arg2 denotes the two operands and result is used to store the result of the expression.

Triples – This representation doesn't make use of extra temporary variable to represent a single operation instead when a reference to another triple's value is needed, a pointer to that triple is used. So, it consist of only three fields namely op, arg1 and arg2.

Indirect Triples – This representation makes use of pointer to the listing of all references to computations which is made separately and stored.

Eg:-

t1 = uminus c

t2 = b * t1

(6)

t3 = uminus c

t4 = b * t3

t5 = t2 + t4

a = t5

#	Op	Arg1	Arg2	Result
(0)	uminus	c		t1
(1)	*	t1	b	t2
(2)	uminus	c		t3
(3)	*	t3	b	t4
(4)	+	t2	t4	t5
(5)	=	t5		a

Quadruple representation

#	Op	Arg1	Arg2
(0)	uminus	c	
(1)	*	(0)	b
(2)	uminus	c	
(3)	*	(2)	b
(4)	+	(1)	(3)
(5)	=	a	(4)

Triples representation

List of pointers to table

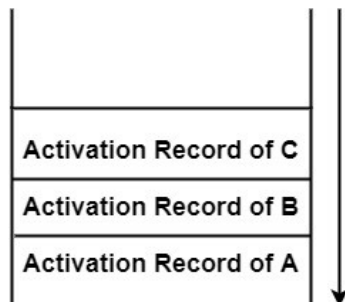
#	Op	Arg1	Arg2
(14)	uminus	c	
(15)	*	(14)	b
(16)	uminus	c	
(17)	*	(16)	b
(18)	+	(15)	(17)
(19)	=	a	(18)

#	Statement
(0)	(14)
(1)	(15)
(2)	(16)
(3)	(17)
(4)	(18)
(5)	(19)

Indirect Triples representation

b) Stack allocation – 4 marks

The stack allocation is a runtime storage management technique. The activation records are pushed and popped as activations begin and end respectively.



Stack Allocation

Heap allocation -4 marks

It enables the allocation of memory in a Non-nested design. Storage can be allocated & freed arbitrarily from an area known as Heap.

(8)

		Head of Free List Initial Free Space List Free Space After Execution Heap Storage Allocation Head of Free List															
		OR															
18	a)	SDD -3 marks <table><tr><th>Production</th><th>Semantic Actions</th></tr><tr><td>$S \rightarrow E$</td><td>$\text{Print}(E.\text{val})$</td></tr><tr><td>$E \rightarrow E_1 + T$</td><td>$E.\text{val} = E_1.\text{val} + T.\text{val}$</td></tr><tr><td>$E \rightarrow T$</td><td>$E.\text{val} = T.\text{val}$</td></tr><tr><td>$T \rightarrow T_1 * F$</td><td>$T.\text{val} = T_1.\text{val} * F.\text{val}$</td></tr><tr><td>$T \rightarrow F$</td><td>$T.\text{val} = F.\text{val}$</td></tr><tr><td>$F \rightarrow \text{digit}$</td><td>$F.\text{val} = \text{digit}.\text{lexval}$</td></tr></table> Annotated parse tree - 4 marks S Value obtained from children nodes E.val =53 E.val = 48 + T.val = 5 T.val = 48 T.val = 6 * F.val = 8 F.val = 6 digit.lexval = 6 digit.lexval = 8 F.val = 5 digit.lexval = 5	Production	Semantic Actions	$S \rightarrow E$	$\text{Print}(E.\text{val})$	$E \rightarrow E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$	$E \rightarrow T$	$E.\text{val} = T.\text{val}$	$T \rightarrow T_1 * F$	$T.\text{val} = T_1.\text{val} * F.\text{val}$	$T \rightarrow F$	$T.\text{val} = F.\text{val}$	$F \rightarrow \text{digit}$	$F.\text{val} = \text{digit}.\text{lexval}$	(7)
Production	Semantic Actions																
$S \rightarrow E$	$\text{Print}(E.\text{val})$																
$E \rightarrow E_1 + T$	$E.\text{val} = E_1.\text{val} + T.\text{val}$																
$E \rightarrow T$	$E.\text{val} = T.\text{val}$																
$T \rightarrow T_1 * F$	$T.\text{val} = T_1.\text{val} * F.\text{val}$																
$T \rightarrow F$	$T.\text{val} = F.\text{val}$																
$F \rightarrow \text{digit}$	$F.\text{val} = \text{digit}.\text{lexval}$																
	b)	Translation scheme – 3 marks Intermediate code - 4 marks	(7)														

		Syntax directed translation scheme for if E then S1 else S2: E.true:= newlabel; E.false:=newlabel; S1.next:=S.next; S2.next:=S.next; S.code:=E.code gen(E.true “:”) S1.code gen(‘goto’ S.next) gen(E.false “:”) S2.code Intermediate code generated: if a>b got L1 goto L2 L1: t1:=inttoreal(b) x:=a+t1 goto L3 L2: t2:=inttoreal(b) x:=a-t2 L3:																									
		Module V																									
19	a)	4 marks for each code optimization technique.	(8)																								
	b)	Three address code – 2 marks <pre> t := a - b u := a - c v := t + u d := v + u </pre> Code sequence – 4 marks <table> <thead> <tr> <th>Statements</th><th>Code Generated</th><th>Register descriptor</th><th>Address descriptor</th></tr> </thead> <tbody> <tr> <td></td><td></td><td>Register empty</td><td></td></tr> <tr> <td>t := a - b</td><td>MOV a, R0 SUB b, R0</td><td>R0 contains t</td><td>t in R0</td></tr> <tr> <td>u := a - c</td><td>MOV a, R1 SUB c, R1</td><td>R0 contains t R1 contains u</td><td>t in R0 u in R1</td></tr> <tr> <td>v := t + u</td><td>ADD R1, R0</td><td>R0 contains v R1 contains u</td><td>u in R1 v in R0</td></tr> <tr> <td>d := v + u</td><td>ADD R1, R0 MOV R0, d</td><td>R0 contains d</td><td>d in R0 d in R0 and memory</td></tr> </tbody> </table>	Statements	Code Generated	Register descriptor	Address descriptor			Register empty		t := a - b	MOV a, R0 SUB b, R0	R0 contains t	t in R0	u := a - c	MOV a, R1 SUB c, R1	R0 contains t R1 contains u	t in R0 u in R1	v := t + u	ADD R1, R0	R0 contains v R1 contains u	u in R1 v in R0	d := v + u	ADD R1, R0 MOV R0, d	R0 contains d	d in R0 d in R0 and memory	(6)
Statements	Code Generated	Register descriptor	Address descriptor																								
		Register empty																									
t := a - b	MOV a, R0 SUB b, R0	R0 contains t	t in R0																								
u := a - c	MOV a, R1 SUB c, R1	R0 contains t R1 contains u	t in R0 u in R1																								
v := t + u	ADD R1, R0	R0 contains v R1 contains u	u in R1 v in R0																								
d := v + u	ADD R1, R0 MOV R0, d	R0 contains d	d in R0 d in R0 and memory																								
		OR																									
20	a)	Input to the code generator Target programs	(7)																								

		Memory management Instruction selection Register allocation Choice of evaluation order Approaches to code generation	
	b)	1. Structure preserving transformations 2. Algebraic transformations (3.5 marks for each)	(7)
